LLMs — Yes or No with Data Science?

What has changed when doing Azure Data Science in the time of LLMs and GenAI?

Tomaž Kaštrun, MVP

About

- Data scientist | BI Developer | Data Analyst
- SQL Server, SAS, R, Python, C#, SAP, SPSS
- 20+years experience MSSQL, DEV, BI, DM, DS
- Frequent community speaker, book author
- Avid coffee drinker & bicycle junkie

http://tomaztsql.wordpress.com
tomaz.kastrun@gmail.com
@tomaz_tsql
/in/tomaztsql
http://github.com/tomaztk
https://mvp.microsoft.com/PublicProfile/5002196

Session material:
https://github.com/tomaztk/Data_science_LLM

SESSION AGENDA

01 Pre-LLM Data Science

How we worked before — classic pipelines, NLP pain, Azure ML Classic

5 min

02 The LLM Revolution

5 key paradigm shifts that changed everyday data science work

10 min

03 Azure AI/ML Evolution

From Designer to AI Foundry — every major service change

10 min

04 Live Demos

Python & R code — EDA, Azure OpenAI, RAG pipelines, Copilot

25 min

05 New DS Workflow

What the modern data scientist's day actually looks like now

5 min

06 Skills, Resources & Q&A

Learning paths, certifications, open questions

5 min

01

Pre-LLM Data Science

How did we work before GenAI changed everything?

The Classic Data Science Pipeline

Before 2022

Common Pain Points Before LLMs:

- NLP required custom tokenizers, embeddings, and massive labeled datasets
- Feature engineering was 70–80% of project time — manual, domain-driven, tedious
- Every new domain meant retraining from scratch — no transfer learning at scale
- Interacting with data meant writing code — no natural language queries
- Automation was limited to AutoML for hyperparameter search, not reasoning tasks

NLP Before LLMs: A Painful Journey

The old way — still relevant context

1. **Collect & label 10,000+ examples**
   Months of annotation work
2. **Preprocess text (tokenize, clean)**
   NLTK, spaCy pipelines
3. **Choose embeddings (Word2Vec, GloVe)**
   Static, no context
4. **Train custom model (LSTM, BERT fine-tune)**
   Days of GPU compute
5. **Evaluate & iterate**
   F1, precision/recall tuning

Task Effort Comparison

Task	Before	After LLM
Sentiment Analysis	Weeks	**Minutes**
Text Classification	2–4 weeks	**Hours**
Named Entity Recognition	Months	**Day**
Question Answering	Research project	**API call**
Summarization	Custom seq2seq	**Prompt**
Language Detection	fastText model	**Zero-shot**

Azure ML — The Classic Era (2018–2021)

Before the GenAI wave

Azure ML Studio (Classic)

Drag-and-drop designer.
No code, limited flexibility.

DEPRECATED

Azure ML SDK v1

Python SDK for experiment tracking & compute clusters.

LEGACY

Azure Cognitive Services

Pre-built vision, speech, language APIs (basic).

EVOLVED

Azure Databricks

Spark-based big data & collaborative notebooks.

STILL STRONG

Azure Data Factory

ETL pipelines for data ingestion and movement.

STILL STRONG

Power BI + AutoML

Embedded AutoML for business analysts.

EVOLVED

02

The LLM Revolution

5 paradigm shifts that changed data science forever

What Are LLMs and Why Do They Matter for DS?

Quick framing

Trained on trillions of tokens

Internet text, code, books — models develop emergent capabilities from scale

In-context learning

No retraining needed — provide examples in the prompt and the model adapts

Instruction following

RLHF-tuned models can follow complex data science instructions naturally

Multimodal extensions

GPT-4V, Gemini — analyze charts, images, and mixed data types

5 Paradigm Shifts LLMs Brought to Data Science

01 NLP Democratized

From weeks of training to a single prompt

02 Code Generation

Co-pilot for data prep, EDA, model tuning

03 Conversational EDA

Ask questions about your data in plain English

04 Feature Engineering

LLMs suggest, generate, and explain features

05 The Copilot Model

AI-augmented DS workflow end-to-end

Shift #1 — NLP Democratized

From specialist skill to commodity

BEFORE LLMs	AFTER LLMs
Custom labeled dataset (1k–100k examples)	Write a clear prompt with 2-3 examples
Tokenization pipeline (NLTK, spaCy, Hugging Face)	Call Azure OpenAI / OpenAI API
Choose embedding: Word2Vec / GloVe / BERT	Get result — adjust prompt if needed
Train / fine-tune for days on GPU	Optionally fine-tune with <100 examples
Evaluate, iterate, re-label	Deploy via managed endpoint
Deploy custom model endpoint	Monitor token cost & latency
Maintain & retrain on domain shift	Switch model version when better one ships

Shift #2 — Code Generation & AI Pair Programming

GitHub Copilot, GPT-4, Azure AI

```
# Prompt to Copilot / ChatGPT
"""
Load the sales CSV, clean missing values,
create lag features for 7 and 30 days,
and train a LightGBM forecasting model.
Return RMSE and feature importance plot."""

import pandas as pd, lightgbm as lgb
from sklearn.metrics import mean_squared_error

df = pd.read_csv('sales.csv')
df['lag_7'] = df['sales'].shift(7)
df['lag_30'] = df['sales'].shift(30)
df.dropna(inplace=True)
```

55%

faster boilerplate code writing

40%

reduction in stack-overflow lookups

3×

faster data prep with LLM assistance

Source: GitHub Copilot user research & Stack Overflow Developer Survey 2024

Shifts #3, #4, #5 — Conversational Data Science

EDA, Features & the Copilot Model

03 Conversational EDA

- "What are the top correlating features with churn?"
- "Explain this distribution to a non-technical stakeholder"
- "Suggest 5 visualizations for this time-series dataset"
- Tools: Azure AI Foundry Playground, ChatGPT Data Analyst

04 LLM-Assisted Features

- "Generate feature ideas from this schema: customer_id, purchase_date, amount"
- "Write a Python function to extract NLP features from support tickets"
- "Explain why this feature has low importance"
- Tools: Azure OpenAI, GitHub Copilot, DSPy

05 The Copilot Model

- Every Azure ML action has a Copilot suggestion
- Notebook generation from dataset descriptions
- Automated experiment explanation & comparison
- Tools: Azure AI Studio Copilot, Fabric Copilot

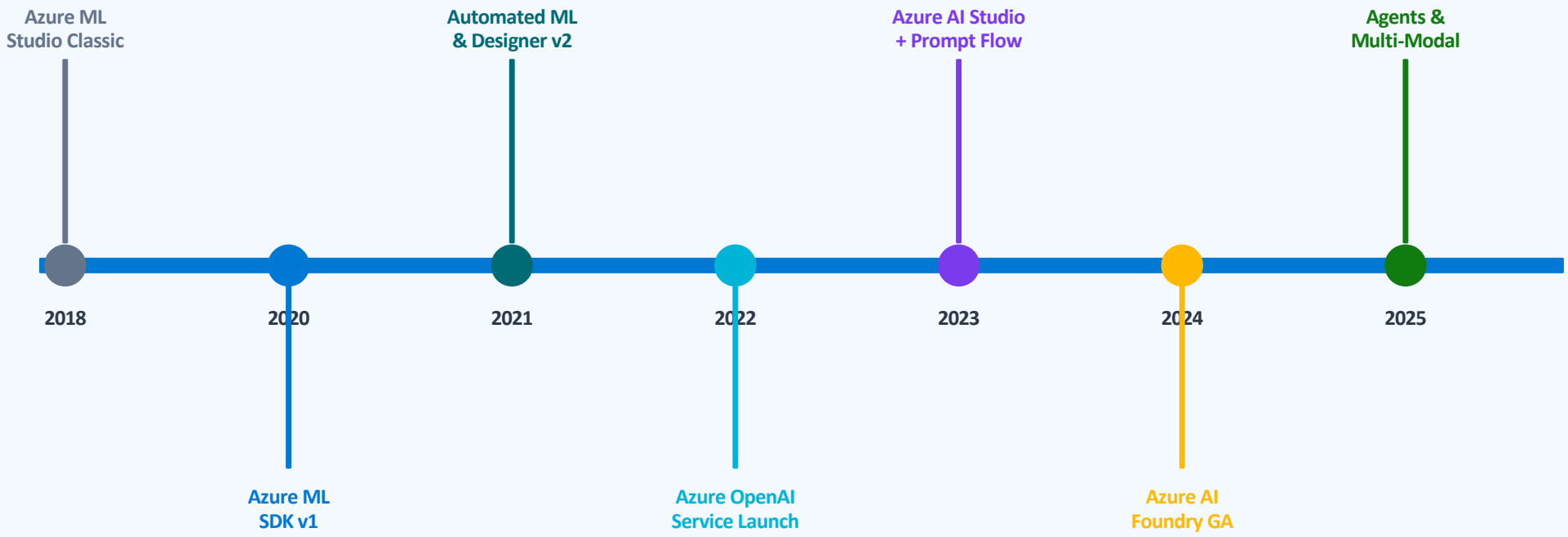
03

Azure AI/ML — Service Evolution

From Azure ML Classic to AI Foundry

Azure AI Services Timeline

How the platform evolved



Key insight: Azure shifted from 'bring your own model' to 'use foundation models + Copilot everything' between 2022–2024

Azure OpenAI Service — Game Changer

Enterprise-grade LLM access on Azure infrastructure

Available Models

- GPT-4o / GPT-4o-mini
- GPT-4 Turbo
- text-embedding-3-large
- DALL-E 3
- Whisper (transcription)
- o1 / o3-mini (reasoning)

Enterprise Features

- Private endpoint (VNet)
- Customer-managed keys
- Data stays in Azure tenant
- Content filtering controls
- Responsible AI dashboard
- SLA: 99.9% uptime

DS Use Cases

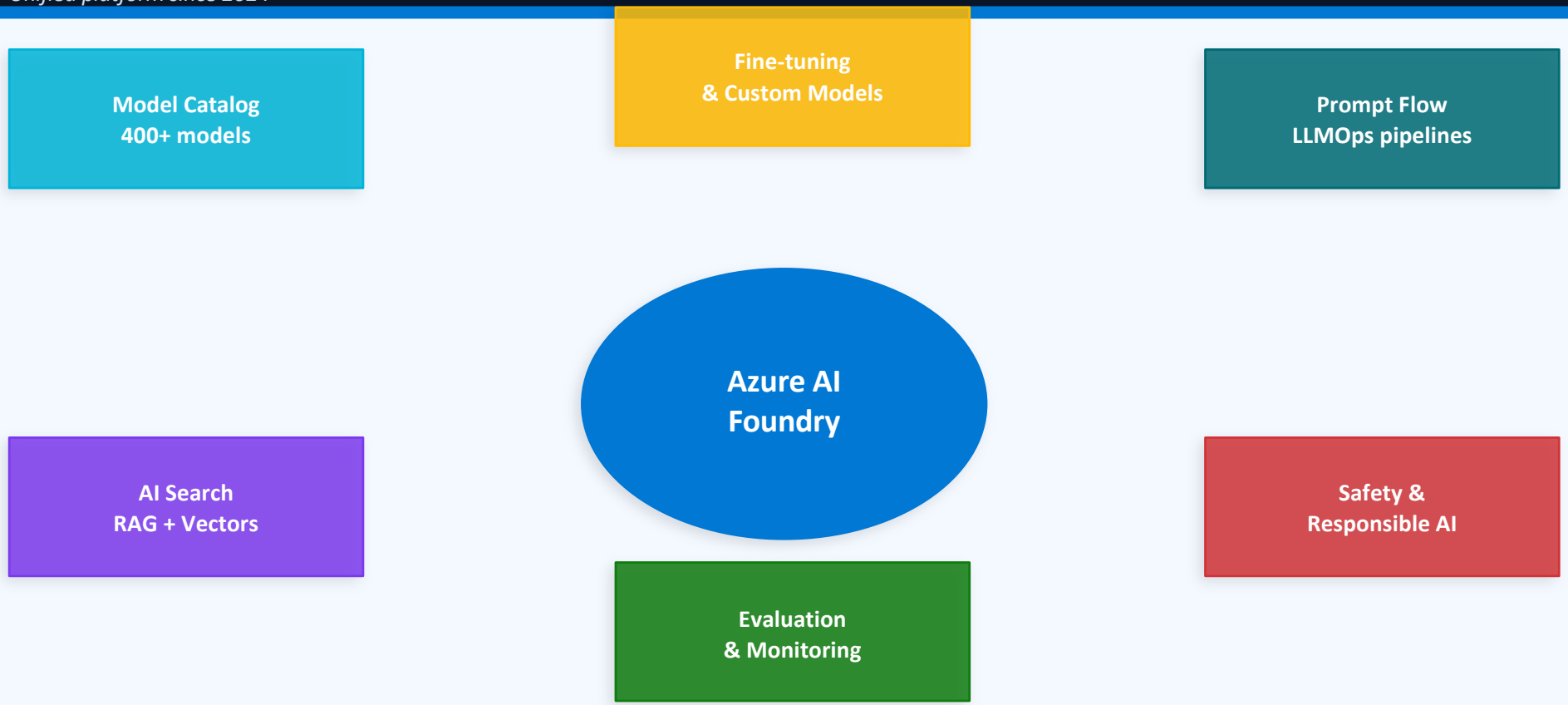
- Chat over tabular data
- Code generation in notebooks
- Structured output (JSON mode)
- Function calling for pipelines
- Embeddings for RAG
- Batch inference jobs

Pricing Model

- Pay-per-token (input + output)
- Provisioned Throughput Units (PTU)
- Batch API 50% discount
- Fine-tuning: training + hosting
- Token caching savings
- Global vs regional routing

Azure AI Foundry — The New Data Science Hub

Unified platform since 2024

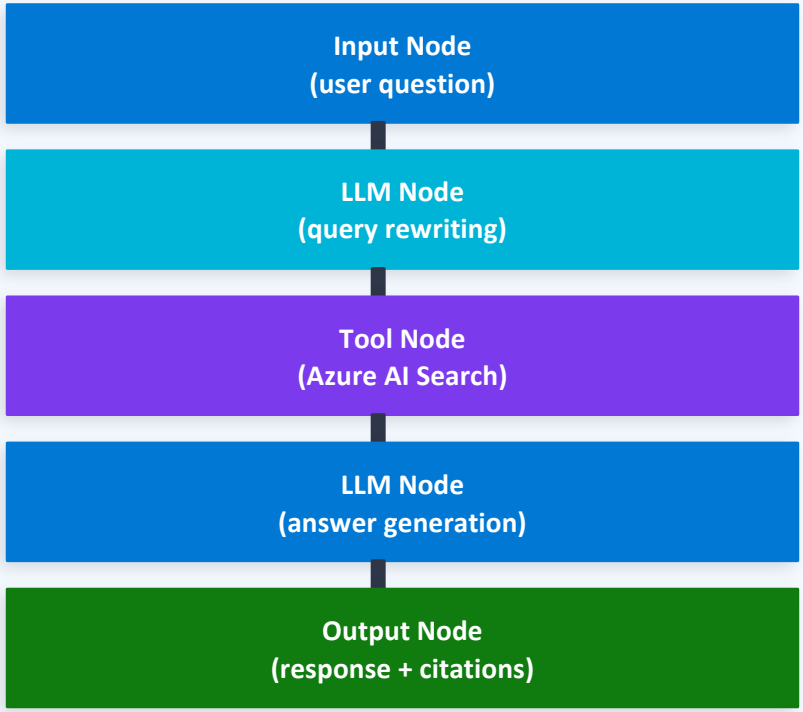


Azure AI Foundry unified Azure ML, Azure Cognitive Services, Azure OpenAI Studio into one platform in 2024

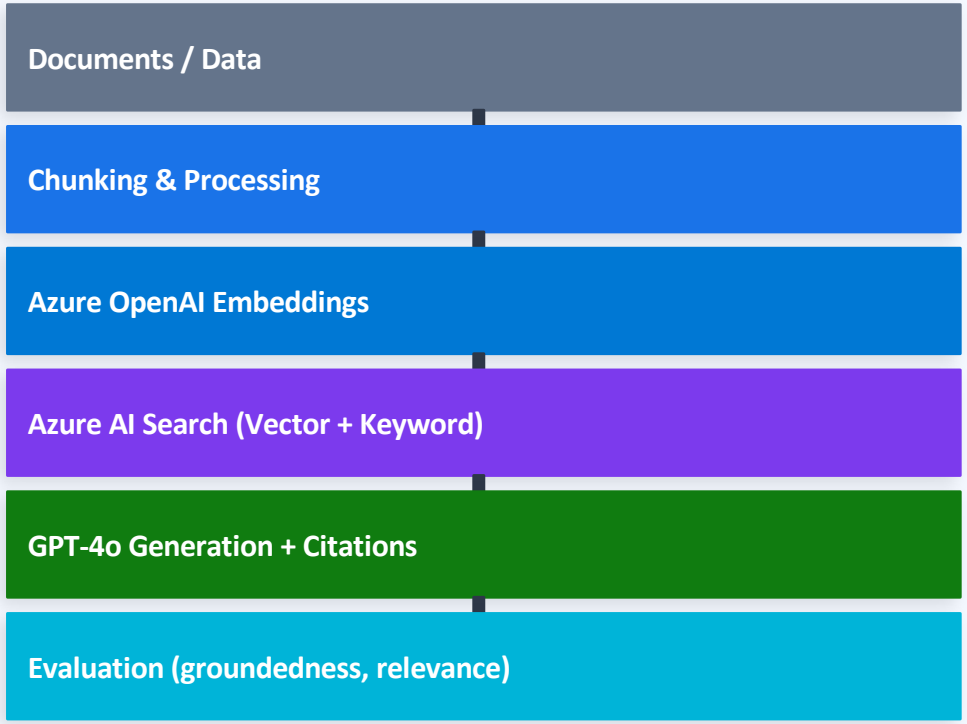
Prompt Flow & RAG — The New MLOps

Building LLM pipelines like ML pipelines

Prompt Flow Pipeline



RAG Architecture on Azure



04

Live Demos

Python, R, Azure OpenAI, RAG, Copilot in action

Demo 1 — Traditional EDA vs LLM-Assisted EDA

Python • demo_01_eda_comparison.py

Traditional (pandas profiling)

```python
import pandas as pd
from ydata_profiling import ProfileReport

df = pd.read_csv("sales_data.csv")
profile = ProfileReport(df,
  title="Sales EDA",
  explorative=True)
profile.to_file("report.html")
#  Takes 2-5 minutes
#  Generates 50-page HTML
#  No natural language insight
```

LLM-Assisted (Azure OpenAI)

```python
from openai import AzureOpenAI
import pandas as pd

df = pd.read_csv("sales_data.csv")
summary = df.describe().to_string()

client = AzureOpenAI(
  azure_endpoint=AZURE_OAI_ENDPOINT,
  api_key=AZURE_OAI_KEY,
  api_version="2024-02-01")

response = client.chat.completions.create(
  model="gpt-4o",
  messages=[{
    "role": "user",
    "content": f"""Analyze: {summary}
    Find anomalies, key patterns,
    suggest 3 ML features."""}])
# Returns in seconds
# Natural language insights
```

Demo 2 — LLM Feature Engineering Assistant

Python • demo_02_feature_engineering.py

Concept: Ask GPT-4o to suggest features from a schema, then auto-generate the code

```
schema = """
customer_id, signup_date, last_purchase_date,
purchase_count, avg_order_value, support_tickets,
plan_type (free/pro/enterprise), region
"""

prompt = f"""
You are a senior ML engineer. Given this schema for a
CHURN PREDICTION model, suggest 10 creative features.
For each feature provide:
1. Feature name (snake_case)
2. Formula or logic
3. Why it predicts churn

Schema: {schema}
Respond as JSON array.
"""

response = client.chat.completions.create(
  model="gpt-4o",
  messages=[{"role": "user", "content": prompt}],
  response_format={"type": "json_object"})

features = json.loads(response.choices[0].message.content)

# Auto-generate implementation code for each feature
for feat in features["features"]:
    code_prompt = f"Write a pandas function for: {feat['name']}: {feat['formula']}"
    code_response = client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": code_prompt}])
```

Result: GPT-4o returns 10 features like 'days_since_last_purchase', 'support_ticket_rate_30d', 'plan_upgrade_velocity' with pandas code

Demo 3 — RAG Pipeline with Azure AI Search

Python • demo_03_rag_pipeline.py

```python
from azure.search.documents import SearchClient
from azure.search.documents.models import VectorizedQuery
from openai import AzureOpenAI
import os

# --- Step 1: Embed the user question ---
embed_client = AzureOpenAI(
    azure_endpoint=os.environ["AZURE_OAI_ENDPOINT"],
    api_key=os.environ["AZURE_OAI_KEY"],
    api_version="2024-02-01")

question = "What are the top churn risk factors for enterprise customers?"

q_embedding = embed_client.embeddings.create(
    model="text-embedding-3-large", input=question
).data[0].embedding

# --- Step 2: Hybrid search (vector + keyword) ---
search_client = SearchClient(
```

Demo 4 — R + Azure OpenAI Integration

R • demo_04_r_llm_integration.R

R users are not left behind — httr2 + tidyverse + Azure OpenAI = powerful DS workflow

```
library(httr2)
library(jsonlite)
library(dplyr)
library(ggplot2)

# Azure OpenAI via httr2
chat_azure <- function(prompt,
    model = "gpt-4o") {
  req <- request(Sys.getenv(
      "AZURE_OAI_ENDPOINT")) |>
    req_url_path_append(
      "openai/deployments",
      model, "chat/completions") |>
    req_url_query(
      "api-version" = "2024-02-01")|>
    req_headers(
      "api-key" = Sys.getenv(
```

```
# Analyze R data with LLM
data(mtcars)
summary_stats <- mtcars |>
  summarise(across(everything(),
    list(mean=mean, sd=sd))) |>
  as.data.frame()

summary_text <- capture.output(
  print(summary(mtcars)))

# Ask GPT-4o to explain findings
insight <- chat_azure(paste(
  "Analyze this R dataset summary",
  "and suggest 3 features for",
  "predicting fuel efficiency (mpg):",
  paste(summary_text, collapse="\n")
))
```

Non-Demo 5 — Microsoft Fabric Copilot & Notebook AI

The integrated Azure DS experience

Fabric Copilot for Notebooks

→ Type '/' in any cell to get AI suggestions

→ "Write a Spark SQL query to find top 10 customers by revenue"

→ Auto-explain existing PySpark code

→ Generate visualizations from data context

Copilot for Data Factory

→ Describe your ETL in English → pipeline generated

→ "Load from SQL Server, pivot by month, write to Delta Lake"

→ Debug pipeline failures with AI explanation

→ Auto-document pipeline steps

SynapseML + OpenAI

→ Native Spark + LLM integration at scale

→ Process millions of rows with LLM transformation

→ Batch classification, summarization, extraction

→ No API rate limit worries — PTU routing

Azure ML AutoML (LLM era)

→ Now includes NLP text classification via fine-tuning

→ Foundation model selection in model catalog

→ Responsible AI explanations via LLM narrative

→ Experiment comparison in natural language

05

The New DS Workflow

What modern data science actually looks like today

Data Science Workflow: Before vs Now

The same project, radically different execution

Phase	Before LLMs	With LLMs (2024–2025)
Problem Framing	Business meetings, manual requirements docs	Copilot helps draft ML problem statement from meeting transcript
Data Discovery	SQL queries + pandas profiling (hours)	Chat with your data via Azure OpenAI + Fabric Copilot (minutes)
Feature Engineering	Domain expert interviews + manual coding (weeks)	LLM suggests features from schema, auto-generates pandas code (hours)
Model Development	Manual sklearn/XGBoost iterations + AutoML	Foundation model fine-tuning + AutoML + Copilot-assisted tuning
NLP Tasks	Custom model training (weeks, labeled data required)	Zero-shot / few-shot prompting via Azure OpenAI (hours)
Model Explanation	SHAP values + manual interpretation	SHAP + GPT-4o natural language explanation for stakeholders
Deployment	Docker + AKS + custom endpoint	Managed online endpoint or Prompt Flow deployment (1-click)

06

Skills, Resources & Q&A

What to learn next and where to find it

Skills & Learning Path for the LLM-Era DS

What to focus on in 2025

Foundation Track

⏱ 2–4 weeks

Core Skills:

- Prompt engineering principles
- Azure OpenAI API (Python/R)
- Embeddings & similarity search
- Azure AI Foundry basics

Certifications:

AZ-900 | AI-900

Data Science Track

⏱ 4–8 weeks

Core Skills:

- RAG pipeline design
- Fine-tuning fundamentals
- LLMOps with Prompt Flow
- Evaluation metrics (RAGAs, etc.)

Certifications:

DP-100 | AI-102

Advanced Track

⏱ 8–12 weeks

Core Skills:

- Agent architectures (AutoGen)
- Multi-modal LLM applications
- SynapseML at Spark scale
- Responsible AI & safety

Certifications:

DP-203 | AZ-305

🔗 learn.microsoft.com/azure/ai-services · microsoft.github.io/promptflow · github.com/Azure-Samples/openai

7

GenAI Chronology & Timeline

From the Transformer paper to Agentic AI — key milestones

The GenAI Timeline: 2017 → 2025

Key model & platform milestones that shaped data science

Foundation Era	Scaling Era	RLHF Era	Agentic Era
2017 Attention is All You Need (Transformer)	2020 GPT-3 175B params	2023 Q1 GPT-4 Llama 1	2024 Q1 Gemini 1.5 Claude 3 Sora
2018 BERT & GPT-1	2021 DALL-E Codex, T5 XXL	2023 Q3 Claude 2 Llama 2 Falcon	2024 Q3 GPT-4o Llama 3 AI Foundry GA
2019 GPT-2 XLNet, RoBERTa	2022 ChatGPT Stable Diffusion		2025 o3 / o4-mini DeepSeek R1 Agentic era

Azure milestones: 2022 → Azure OpenAI Preview · 2023 → AI Studio · 2024 → AI Foundry GA + Model Catalog · 2025 → Agent Service GA

What Each Era Meant for Data Scientists

GenAI impact by milestone group

Foundation Era (2017–2019)

- Transfer learning became viable for NLP tasks
- BERT fine-tuning replaced custom NLP pipelines
- Contextual embeddings > Word2Vec/GloVe
- Azure: Cognitive Services Text Analytics v3

Scaling Era (2020–2022)

- GPT-3 showed few-shot learning — no labels needed
- Code generation emerged (GitHub Copilot, Codex)
- ChatGPT crossed into mainstream in 6 days
- Azure: OpenAI Service launched (public preview Nov 2022)

RLHF Era (2023)

- Instruction following became reliable and safe
- Multimodal (GPT-4V) — analyse charts & images
- Open-source models (Llama 2) changed deployment
- Azure: AI Studio, Prompt Flow, RAG patterns

Agentic Era (2024–2025)

- Agents plan & execute multi-step DS pipelines
- LLMs evaluate other LLMs (LLM-as-judge)
- Reasoning models (o1/o3) for math/stats problems
- Azure: AI Foundry GA, Agent Service, Model Catalog 1800+

Demo A — GenAI Timeline Explorer (Interactive)

Python • demo_A_genai_timeline.py

Generates an interactive Plotly timeline + asks GPT-4o to explain the impact of each milestone on your specific DS domain

```
import plotly.express as px, pandas as pd
from openai import AzureOpenAI

milestones = pd.DataFrame([
  {"year":2017,"event":"Transformer","era":"Foundation","impact":3},
  {"year":2018,"event":"BERT","era":"Foundation","impact":4},
  {"year":2020,"event":"GPT-3","era":"Scaling","impact":5},
  {"year":2022,"event":"ChatGPT 🚀","era":"RLHF","impact":10},
  {"year":2023,"event":"GPT-4 / Claude 2","era":"RLHF","impact":8},
  {"year":2024,"event":"AI Foundry GA","era":"Agentic","impact":7},
  {"year":2025,"event":"Agent Service","era":"Agentic","impact":9},
])

fig = px.scatter(milestones, x="year", y="impact", size="impact",
  color="era", text="event", size_max=60,
  title="GenAI Milestones — Impact on Data Science",
  color_discrete_map={"Foundation":"#0078D4","Scaling":"#1E8449",
```

8

How IDEs Have Evolved

From Jupyter notebooks to AI-native development environments

The IDE Evolution for Data Scientists

Pre-LLM vs Post-LLM development experience

Pre-LLM Era IDEs

Jupyter Notebook
Still popular

Classic notebook — cells, markdown, plots

RStudio
Still popular

Gold standard for R — console + plots + help

VS Code (basic)
Evolved hugely

Polyglot editor, Python ext, Pylance

PyCharm
Still used

Professional Python IDE, debugger, profiler

LLM-Native IDEs

GitHub Copilot
Game changer

Inline AI completions — works in all IDEs

VS Code + Copilot
New standard

Chat, explain, fix, generate — AI sidebar

Cursor IDE
Fast growing

GPT-4 native — entire codebase context

Azure AI Foundry Playground
Azure native

Prompt Flow + model testing + tracing

Fabric Notebooks + Copilot
Azure native

GitHub Copilot & AI IDEs in DS Practice

What AI-native coding means day-to-day for data scientists

Autocomplete → Intent Complete

✗ Tab-complete one token at a time

→ ✓ **Type a comment → full function generated**

Stack Overflow → Inline Chat

✗ Leave IDE → search → copy-paste → adapt

→ ✓ **Ctrl+I → ask in natural language → done**

Documentation → /explain

✗ Read docs, infer parameters

→ ✓ **/explain this pandas groupby chain**

Manual Tests → /tests

✗ Write pytest by hand

→ ✓ **/tests for this feature function**

Azure ML SDK → Copilot-guided

✗ Memorise SDK v2 job schema

→ ✓ **Describe job in English → SDK code generated**

Demo B — IDE Copilot Productivity Benchmark

Python • demo_B_ide_copilot.py

Simulates Copilot-style code generation via API: intent comment → full DS function → test generation → time comparison

```
# INTENT-DRIVEN CODING — simulating GitHub Copilot via Azure OpenAI
intents = [
  "# Load a CSV, clean missing values, encode categoricals, split 80/20",
  "# Compute RMSE and MAE between y_true and y_pred arrays",
  "# Plot feature importance from a fitted LightGBM model",
  "# Run k-fold CV with stratified splits and return mean AUC",
]

import time; from openai import AzureOpenAI
client = AzureOpenAI(azure_endpoint=ENDPOINT, api_key=KEY, api_version="2024-02-01")

for intent in intents:
    t0 = time.time()
    resp = client.chat.completions.create(
        model="gpt-4o",
        messages=[{
          "role":"system",
```

9

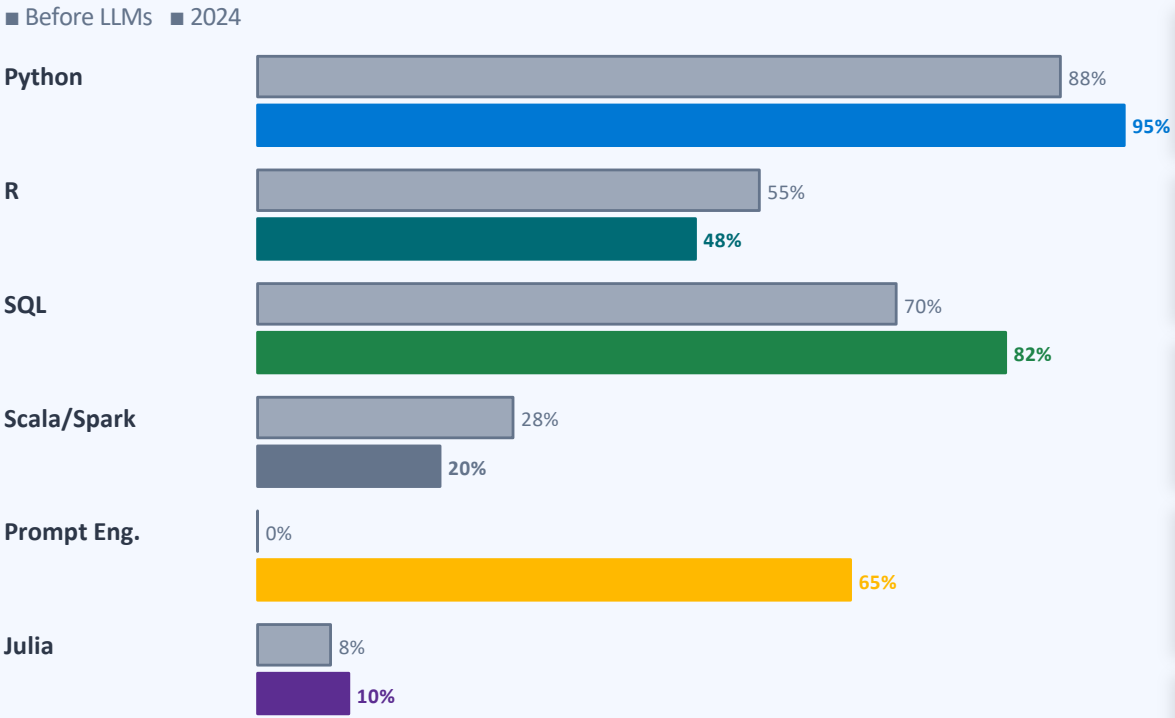
Programming Languages

How language choices shifted in the LLM era

Programming Language Trends in Data Science

What changed, what stayed, what emerged

DS Language Usage (% of practitioners)



Key Shifts Explained

Python ↑ Dominant

LLM SDKs (openai, langchain, azure-ai) are Python-first. Prompt Eng. = Python strings.

R → Stable

httr2 covers Azure OpenAI well. Posit doubled down. Quarto unifies R+Python.

SQL ↑ Elevated

Natural language → SQL (Text-to-SQL) made SQL more relevant, not less. Azure Fabric Copilot.

Prompting ⭐ New

Prompt engineering is a first-class skill. Structured prompts = programs. DSPy, LangChain.

Rust/Go ↑ Infra

LLM serving infra (vLLM, Candle) uses Rust. Most DS don't need it but it matters for MLOps.

Prompting as a Programming Paradigm

The new 'language' every data scientist needs

Zero-Shot

No examples. Just the task description. Works for classification, extraction, summarisation.

```
"Classify this text as
Positive/Negative/Neutral: {text}"
```

Few-Shot

2–5 labelled examples in the prompt. Often beats fine-tuned models for rare classes.

```
"Text: great product! → Positive
Text: terrible experience → Negative
Text: {input} → "
```

Chain-of-Thought

Ask for reasoning before the answer. Dramatically improves math and logic tasks.

```
"Solve step by step. Think through it first,
then give the final answer."
```

Structured Output

JSON mode + response_format. Turns LLM into a typed function. Essential for DS pipelines.

```
response_format={"type":"json_object"}
```

Azure Prompt Flow

DAG-based prompt programs. Version-controlled, evaluated, deployed like ML models.

```
"flow.dag.yaml + LLM nodes + Python tools +
eval node"
```

DSPy / LangChain

Frameworks that treat prompts as learnable parameters. Auto-optimise prompt via data.

```
"program = dspy.ChainOfThought('question ->
answer')"
```

Demo C — Multi-Language LLM Integration

Python + R + SQL • demo_C_languages.py

Shows the same DS task solved in Python, R (via httr2), and SQL (Text-to-SQL via Azure OpenAI)

```
# PYTHON: Sentiment classification — zero-shot
resp = client.chat.completions.create(model="gpt-4o",
    messages=[{"role":"user","content":f"Classify: {text} → Positive/Neutral/Negative"}])

# SQL (Text-to-SQL via Azure OpenAI)
schema = "orders(id,customer_id,amount,date), customers(id,name,plan)"
prompt = f"Write SQL: top 10 customers by total revenue last 90 days\nSchema: {schema}"
sql_resp = client.chat.completions.create(model="gpt-4o",
    messages=[{"role":"user","content":prompt}])
generated_sql = sql_resp.choices[0].message.content
# → SELECT c.name, SUM(o.amount) AS revenue FROM orders o
#   JOIN customers c ON o.customer_id = c.id
#   WHERE o.date >= CURRENT_DATE - 90
#   GROUP BY c.id ORDER BY revenue DESC LIMIT 10

# STRUCTURED OUTPUT — JSON mode for pipeline-safe parsing
resp = client.chat.completions.create(model="gpt-4o",
```

10

Data Visualisation

How we present data changed with AI — from code to conversation

Data Visualisation Tools: Before & After LLMs

The shift from code-first to intent-first charting

Tool	Before LLMs	After LLMs / Azure Copilot	Trend
matplotlib	Verbose, 20+ lines for a styled chart	Still used — Copilot writes it for you in 3s	→
seaborn	Statistical plots, manual styling	Copilot-generated seaborn in natural language	→
ggplot2 (R)	Grammar of graphics — powerful but verbose	LLM generates full ggplot2 from description	→
Plotly/Dash	Interactive charts, heavy JS knowledge	Describe chart → Plotly code generated	↑
Power BI	Drag-and-drop, DAX formulas manually	Copilot: 'Show revenue by region last year'	↑↑
Azure AI Vision / GPT-4V	N/A — charts were opaque to machines	GPT-4o reads your chart & explains it!	⭐ **NEW**
Observable / Vega-Lite	Niche — requires JS knowledge	LLMs generate Vega-Lite specs from intent	↑

Three New Visualisation Superpowers

What LLMs added to the data viz toolkit

1 Natural Language → Chart Code

Describe a chart in English → get matplotlib/ggplot2/Plotly code instantly. "Plot a dual-axis line chart showing revenue and churn rate by month, styled for a board presentation."

Azure: *Power BI Copilot, Fabric Notebook Copilot, Azure OpenAI code gen*

2 Chart → Natural Language (Multimodal)

Upload a chart image to GPT-4o and ask it to explain trends, identify anomalies, or write the title and axis labels for a non-technical audience.

Azure: *GPT-4o via Azure OpenAI (vision endpoint), Azure AI Vision Describe Image*

3 Automated Viz Narration

Generate slide-ready bullet-point commentary from chart data. Stakeholders receive the chart + a paragraph explaining what it means — auto-generated every time the data refreshes.

Azure: *Power BI + Copilot narrator, Azure OpenAI + Prompt Flow scheduled pipeline*

Demo D — LLM-Powered Visualisation Generation

Python • demo_D_visualisation.py

NL → chart code + chart → narrative + GPT-4V chart explanation. Three patterns in one demo.

```
# PATTERN 1: Natural Language → Matplotlib code
nl_request = """
Create a matplotlib figure with 2 subplots side-by-side:
Left: bar chart of monthly revenue Jan-Dec with error bars.
Right: line chart showing 30-day rolling churn rate.
Use a clean, professional style suitable for board presentation.
"""
resp = client.chat.completions.create(model="gpt-4o",
    messages=[{"role":"user","content":"Write Python matplotlib code: " + nl_request}])
exec(resp.choices[0].message.content)  # renders the chart!

# PATTERN 2: Chart → Narrative (attach chart as base64 image)
import base64
with open("chart.png","rb") as f:
    img_b64 = base64.b64encode(f.read()).decode()

resp = client.chat.completions.create(model="gpt-4o",
```

11

Algorithms

What happened to classical ML algorithms in the GenAI era?

Algorithm Landscape: Classical ML vs GenAI Era

Nothing is dead — roles have changed

Algorithm	Cat.	2024 Verdict	What changed
Linear / Logistic Regression	Classical	↑ **Still baseline**	Baseline, explainability, small data — LLMs can't beat it here
Decision Trees / Random Forest	Classical	↑ **Still core**	Tabular data king. LightGBM/XGBoost still dominate kaggle competitions
SVMs	Classical	→ **Niche**	Rare now. Kernel SVMs replaced by GBMs/neural nets for most tasks
LSTM / RNNs	Deep	↓ **Mostly replaced**	Transformers outperformed RNNs on all sequential tasks. Still used in edge devices.
CNNs	Deep	→ **Image-specific**	Still best for pure vision tasks. ViT challenging but CNNs remain efficient.
Transformers	Deep	↑↑ **Dominant**	The algorithm of the era. Attention mechanism powers all frontier LLMs.
Foundation Models (GPT, BERT…)	GenAI	☆ **Paradigm shift**	Pre-train once, fine-tune or prompt for anything. Replaces per-task training.
RAG / Retrieval Models	GenAI	☆ **New pattern**	Non-parametric knowledge — combine retrieval + generation. Azure AI Search.
Embedding Models	GenAI	↑↑ **Critical infra**	text-embedding-3-large powers search, RAG, clustering, dedup at scale.

Algorithm Selection in 2025: Decision Framework

Classical ML vs Fine-tuning vs RAG vs Zero-shot

Demo E — Algorithm Selection Assistant

Python • demo_E_algorithms.py

GPT-4o as an algorithm advisor: describe your problem → get ranked algorithm recommendations with code templates

```
PROBLEM = """
Dataset: 50,000 customers, 25 features (mostly numeric, 3 categorical).
Target: binary churn (7% positive class).
Constraints: prediction must be explained to non-technical stakeholders.
Priority: interpretability > raw AUC.
Azure ML compute: Standard_DS3_v2 cluster (4 cores).
"""


resp = client.chat.completions.create(model="gpt-4o",
    messages=[{"role":"system","content":"You are an expert ML engineer. Given a problem description, recommend the top 3 algorithms with rationale and starter code."},
              {"role":"user","content":f"Recommend algorithms for: {PROBLEM}"}],
    response_format={"type":"json_object"})


recommendations = json.loads(resp.choices[0].message.content)
# Returns: [{algorithm, rationale, pros, cons, azure_service, starter_code}, ...]


for i, rec in enumerate(recommendations["algorithms"], 1):
```

12

Interpretation & Explainability

From SHAP plots to natural language explanations

Explainability (XAI): Tools & Techniques

Classical XAI extended by LLM narrative layers

SHAP Values

Classical

Shapley-based feature attribution. Gold standard for tabular models. Still essential.

Azure: *Azure Responsible AI dashboard*

LIME

Classical

Local surrogate model. Explains individual predictions. Slower than SHAP.

Azure: *Responsible AI Insights (SDK)*

Partial Dependence Plots (PDP)

Classical

Marginal effect of features. Good for communicating monotonic relationships.

Azure: *Azure ML Studio Explain tab*

Integrated Gradients

Deep

Attributions for neural network predictions. Required for CNNs/Transformers.

Azure: *Responsible AI SDK*

LLM Narration of SHAP

GenAI

GPT-4o turns SHAP bar charts into English paragraphs for business stakeholders.

Azure: *Azure OpenAI + custom pipeline*

AI Foundry Groundedness

GenAI

Evaluate if LLM answers are grounded in retrieved context. Prevents hallucination.

Azure: *Azure AI Foundry Evaluation*

Responsible AI Dashboard

Azure

Unified error analysis, fairness, causal inference, and counterfactuals in one tool.

Azure: *Azure ML Responsible AI Dashboard*

Prompt Flow Tracing

Azure

Full trace of every LLM call, tool invocation, token, and latency in a pipeline run.

Azure: *Azure AI Foundry Prompt Flow*

Azure Responsible AI + LLM Evaluation Metrics

How evaluation changed in the GenAI era

Classical ML Evaluation	GenAI / LLM Evaluation (New!)	
AUC-ROC / RMSE / F1	**Groundedness**	Is answer supported by context? (RAG)
Confusion matrix	**Relevance**	Does answer address the question?
SHAP global importance	**Coherence**	Is the response logically consistent?
PDP / ICE plots	**Fluency**	Is the language natural and readable?
Fairness: demographic parity	**LLM-as-Judge**	GPT-4o scores GPT-4o outputs at scale
Error analysis cohorts	**Token cost / latency**	New operational efficiency metrics

Azure ML Responsible AI Dashboard

Combines error analysis + SHAP + fairness + counterfactuals + causal analysis in one dashboard. Now includes LLM groundedness metrics for GenAI models.

Demo F — SHAP + LLM Explainability Pipeline

Python • demo_F_explainability.py

SHAP values → GPT-4o → stakeholder narrative + Azure RAI dashboard integration + LLM-as-judge evaluation

```
import shap, lightgbm as lgb
from openai import AzureOpenAI

# 1. Train model + compute SHAP
model = lgb.LGBMClassifier().fit(X_train, y_train)
explainer = shap.TreeExplainer(model)
shap_values = explainer(X_test)

# 2. For a specific prediction, build SHAP summary
customer_idx = 42
shap_row = dict(zip(X_test.columns, shap_values.values[customer_idx]))
top_factors = sorted(shap_row.items(), key=lambda x: abs(x[1]), reverse=True)[:5]

factor_text = "\n".join([
    f"  {feat}: SHAP={val:+.3f} ({'risk driver' if val>0 else 'protective'})"
    for feat, val in top_factors])
```

Key Takeaways

What to remember from today

1

Yes — LLMs have fundamentally changed data science

NLP is democratized, code generation is real, and EDA is conversational. The workflow has shifted from months to days for many tasks.

2

Azure AI/ML has undergone a major transformation

Azure AI Foundry unifies ML + GenAI. Prompt Flow is the new MLOps. Azure OpenAI is enterprise-ready with compliance guarantees.

3

The core DS skills still matter more than ever

Statistics, domain understanding, data quality, and model evaluation are still your foundation — LLMs amplify them, not replace them.

4

RAG > Fine-tuning for most enterprise use cases

Start with RAG + Azure AI Search. Fine-tune only when you need to change behavior or reduce latency at scale.

5

R and Python users both have clear paths forward

httr2 for R, openai SDK for Python — both communities have LLM integration patterns that fit existing workflows.

Questions & Discussion

LLMs — Yes or No with Data Science?
The answer is: Yes, and here's how to start today.

Thank you for attending!